Universidad de Magallanes

Informe de Proyecto N.º 2

Integrantes:
Martina Márquez
Eban Delgado
Yanira Mansilla

Fecha: 30 de octubre de 2026

Asignatura: Estructura De Datos
Profesor: Jacqueline Aldridge Águila

Índice

1. Introducción **1**

2. Objetivos **2**

2.1. Objetivos Especificos 2

3. Marco teórico **3**

4. Desarrollo Del Proyecto **4**

4.1. Implementacion de lista simple: módulos Inventario y objetos 4

4.2. Implementación de pilas: módulo historial 8

4.3. Implementaciion de colas: modulo combate 14

4.4. Implementacion del modulo aventura 15

4.5. Implementacion del modulo interfaz 18

4.6. Implementación del módulo personajes 20

5. Problemas que se dieron en el proyecto **22**

6. Pruebas de ejecucion y resultados **23**

7. Conclusiones **24**

8. Referencias **25**

9. Anexos **26**

9.1. Contribucion de cada estudiante 26

9.2. Declaracion de IA 26

9.3. Declaración de IA: Declaración individual 27

1. Introducción

En el presente proyecto se desarrolló un programa en lenguaje C que simula el funcionamiento de un juego RPG, el sistema central de este proyecto gira entorno a un personaje, el cual explora 3 lugares con diferentes dificultades e interactua con diferentes objetos llevando cabo acciones fundamentales como la recolección de objetos, la administración de un inventario y el enfrentamiento contra diversos enemigos, todo esto ocurre en la terminal.

Desde el punto de vista de codigo, esta desarrollado en Lenguaje C, se aplicaron listas enlazadas, pilas y colas para gestionar cada uno de los modulos del juego, esto significa que entidades como los personajes, los objetos del inventario, los grupos de enemigos durante el combate y el registro de historial de acciones operan sobre modelos dinámicos de memoria. Por lo tanto, el desarrollo no solo busca simular un entorno realista, sino también asegurar un manejo estricto de la memoria que contemple escenarios complejos como estructuras vacías, elementos únicos, falta de memoria y la correcta eliminación de nodos para evitar fugas de recursos.

2. Objetivos

Diseñar e implementar un sistema funcional de un juego de rol basado en lenguaje C, aplicando y construyendo listas enlazadas, pilas y colas, para gestionar de manera eficiente las distintas entidades y mecánicas del juego.

2.1. Objetivos Especificos

- **Desarrollar y gestionar** correctamente las estructuras base para personajes, objetos, enemigos y acciones, asegurando un buen control de la memoria ante casos límite y operaciones inválidas ingresadas por el usuario.
- **Implementar** un módulo de aventura e inventario que permita la exploración del personaje, la recolección de ítems mediante eventos generados por pasos y la manipulación completa de estos elementos, como equiparlos, usarlos o descartarlos.
- **Estructurar** un sistema de combate secuencial donde el jugador enfrente a múltiples enemigos siguiendo un orden de aparición, requiriendo el uso de estructuras de datos adecuadas para administrar los turnos y la permanencia de los enemigos en la batalla.

3. Marco teórico

Este proyecto se basa en construir nuestras propias formas de organizar la información del juego (como listas, pilas y colas). Estas formas de organizar los datos son esenciales para que el juego use la memoria del computador de manera correcta mientras se está jugando. Para que el sistema funcione bien, es clave entender cómo debe comportarse cada una de estas estructuras, asegurándonos de que la forma en que guardamos, ordenamos y sacamos la información cumpla exactamente con las reglas de cada una.

Para manejar el inventario, se utilizó una "lista enlazada"que cuenta con un punto de inicio especial, lo que permite ir conectando un objeto detrás de otro y facilita agregar o quitar cosas durante la aventura. Para los combates, se programó una cola para organizar a los enemigos, asegurando que ataquen exactamente en el mismo orden en que van apareciendo, siguiendo la regla de el primero en formarse es el primero en pelear. Por último, para recordar los pasos del jugador, se implementó una "pila.[en] el historial; esto guarda las acciones una encima de la otra, logrando que la acción más reciente quede siempre en la cima para poder leerla fácilmente o deshacerla si es necesario.

Cada una de estas formas de agrupar datos tiene sus propias reglas y acciones que se respetan estrictamente. En el inventario se crearon funciones específicas para buscar un objeto por su número de identificación, mostrar todo lo que hay guardado en pantalla o eliminar un ítem en particular. En la fila de combate, se programaron acciones para agregar un enemigo a la espera y para sacarlo de la fila cuando es su turno de pelear o es derrotado. En cuanto al historial, la pila permite registrar un evento nuevo empujándolo hacia la cima o eliminar el último registro en caso de que el jugador decida deshacer la acción de descartar un objeto.

Un punto importante en el desarrollo de este proyecto es cómo el código cuida la memoria del computador para evitar que el juego se cierre de golpe. En lugar de reservar un bloque gigante de memoria por si acaso, el programa le pide prestado al computador un espacio a la medida exacta solo cuando lo necesita, para asignar el tamaño justo al crear un personaje, un enemigo o un objeto. Además, el código cuenta con protección contra el vacío, siempre revisa si las agrupaciones están vacías. Esto evita que el sistema colapse al intentar buscar algo que no existe. Por último, el juego prevé el escenario en el que el computador se quede sin memoria RAM disponible. Cuando los elementos ya no sirven, la memoria se limpia y se devuelve, manteniendo un orden estructural que, además, hace posible escribir todo el progreso en un archivo de texto para guardar y cargar la partida sin errores.

4. Desarrollo Del Proyecto

4.1. Implementacion de lista simple: módulos Inventario y objetos

Una lista enlazada se rige por un principio de secuencia por referencias a diferencia de los arreglos, sus elementos los nodos no se almacenan en posiciones de memoria contiguas, sino que cada uno contiene el dato y un puntero que direcciona explícitamente al siguiente elemento, permitiendo una asignación dinámica y flexible de la memoria.

Todas as listas simples tienen algo en comun y eso seria unas funciones base, las cuales son:

- MakeEmpty: crea el nodo centinela.
- IsEmpty: indica si la lista está vacía.
- Insert: Inserta un elemento a la lista.
- Remove: Remueve un elemento de la lista.
- Search: Busca un elemento de la lista.
- Display: Muestra el contenido de la lista.

El modulo de Inventario: fue tomando como referencia el trabajo entregado por Milton, esto debido a que en el trabajo entregado la instruccion era ïmplementar las funciones a un inventario", con eso dicho en ese trabajo entregado lo que nos pedian era a traves de una lista enlazada doble, mientras que en este trabajo se implemento una lista enlazada simple.

Asi que para aplicarlo de forma correcta se transformaron las funciones que ya existian en unas que se adpaten a la lista enlazada simple, para esto se deshizo de las funciones que insertaban datos en cualquier posicion que no sea el ultimo, tambien las listas de desplazamiento dentro de la lista y las de localizacion segun posicion, ya que estas no tienen una utilidad real ya que dentro de la ejecucion del inventario uno pone sus parametros para encontrar el objeto y la lista avanza hasta encontrar lo buscado, y en caso de que no encuentre el objeto muestra por pantalla un mensaje que dice .ºbjeto no encontrado en el inventario."

Tomando esto en consideracion despues de la adaptacion a lista enlazada simple quedaron las siguientes funciones para el funcionamiento, que funcion de la lista simple cumple, y el como fueron estructuradas en codigo (cabe recalcar que las funciones en codigo fueron hechas con snake case y no con camel case):

- CrearInventarioVacio: InvMakeEmpty. Esta funcion crea el nodo centinela del inventario.
- InsertarObjetoenelInventario: InvInsert. Esta funcion inserta un objeto al inventario.
- BorrarObjetodelInventario: InvDelete. Esta funcion borra un objeto del inventario.
- EstaVacioElInventario: InvIsEmpty. Esta funcion verifica si el inventario esta vacio.
- DestruirInventario: InvDestry. Esta funcion borra el inventaro.
- MostrarInventario: InvDisplay. Esta funcion muestra el inventario completo.
- BuscarporID: InvSearchById. Esta funcion busca un objeto en base a su id.
- BuscarporNombre: InvSearchByName. Esta funcion busca un objeto en base a su nombre.

Estas funciones toman como parametros variables de la estructura Inventario inicializada precisamente para este caso, ademas de valores enteros y strings.

Estructura Inventario

```
typedef struct Node* PtrToNode;
typedef PtrToNode Inventario;
typedef PtrToNode InventarioElement;

struct Node {
    Objeto objeto;
    PtrToNode next;
};
```

El modulo Objetos: fue hecho tomando como referencia tambien al trabajo entregado por Milton, con la diferencia de que este no esta hecho tomando como referencia la instruccion del trabajo, si no las funciones del objeto con las que uno debe trabajar al realizar el trabajo, siendo estas funciones en relacion a los productos, que en este trabajo fueron transformados a objetos.

Aqui las funciones que se usaron en el proyecto, si es que estan basadas en las funciones de milton, y en caso de que si en cual estan basadas (cabe recalcar que las funciones en codigo fueron hechas con snake case y no con camel case):

- CrearObjeto: Basada en la funcion ProdCreate, esta funcion crea el producto/objeto.

```
Objeto Crear_Objeto(int id, const char* nombre, int tipo,
    int valor_efecto)
{
    Objeto obj;
    obj.id=id;
    obj.tipo=tipo;
    obj.valor_efecto=valor_efecto;

    obj.nombre=malloc(strlen(nombre) + 1);
    if (obj.nombre==NULL)
    {
        printf("Error: Memoria insuficiente para crear el
            objeto.\n");
        obj.id=-1;
        return obj;
    }
    strcpy(obj.nombre, nombre);

    return obj;
}
```

- MostrarObjeto: Basada en la funcion ProdPrint, esta funcion muestra por pantalla el produco/objeto.

```
void Mostrar_Objeto(Objeto obj)
{
    printf("[%d] %s (Tipo: %d, Efecto/Valor: %d)\n", obj.
        id, obj.nombre, obj.tipo, obj.valor_efecto);
}
```

- BorrarObjeto: Basada en la funcion ProdDelete, esta funcion borra un producto/objeto de la lista y su espcio de memoria.

```
void Borrar_Objeto(Objeto obj)
{
    if (obj.nombre != NULL)
    {
        free(obj.nombre);
    }
}
```

- DatosObjetos: Esta funcion no esta basada en nada, su funcionalidad es almacenar los datos de objetos y crearlos llamando a la funcion CrearObjetos.

```
    Objeto datos_objetos(int obj)
    {
        Objeto objeto;

        if(obj<=40){
            objeto=Crear_Objeto(1,"Espada corta de cobre"
               ,1,10);
        }
        else if(obj<=70){
            objeto=Crear_Objeto(2,"Arco primitivo",1,10);
        }
        else if(obj<=85){
            objeto=Crear_Objeto(3,"Pocion curativa menor"
               ,2,2);//tipo 2 es un consumible de cura, y el
               ultimo valor es lo que curara, asi que
               cambiara  segun la vida maxima
        }
        else if(obj<=95){
            objeto=Crear_Objeto(4,"Espada larga de hierro"
               ,1,25);
        }
        else if(obj<=100){
            objeto=Crear_Objeto(5,"Cota de malla",3,8);
        }

        return objeto;
    }
```

Estas funciones toman como parametros variables de la estructura Objetos inicializada precisamente para este caso, ademas de valores enteros y strings.

```
typedef struct _Objeto {
    int id;
    char* nombre;
    int tipo;
    int valor_efecto;
} Objeto;
```

4.2. Implementación de pilas: módulo historial

Las pilas poseen un principio de "Last In, First Out"(LIFO), donde el ultimo elemento que se añade a la pila es el primero en ser retirado.

Tomando en consideración que el historial debe de registrar las acciones realizadas durante la partida mediante mensajes descriptivos y genericos, además de permitir que se pueda deshacer la ultima accion si esta fue del tipo descartar objeto (equiparlo o consumirlo).

Con la descripción de la estructura historial y los principios del dato abstracto, las pilas fueron la opción que mas se acomodaba a esta estructura. Debido a que el historial no tenia un maximo en que tanto podia guardar, la mejor forma de implementar esta pila era mediante lista enlazada simple mas que un arreglo fijo para facilitar la insercion y eliminacion de elementos. Las pilas siguen el principio *Last In, First Out* (LIFO): el último elemento que se añade es el primero en ser retirado.

El historial debe registrar las acciones realizadas durante la partida mediante mensajes descriptivos y genéricos. Además, debe permitir deshacer la última acción cuando esta corresponde al uso de un objeto (equiparlo o consumirlo), devolviéndolo al inventario y revirtiendo su efecto sobre el personaje.

Dadas estas características, la pila fue la estructura que mejor se acomodó al problema, pues siempre se consulta o modifica la acción más reciente. Como el historial no tiene un máximo de acciones, se implementó mediante una lista enlazada simple con nodo centinela, en lugar de un arreglo de tamaño fijo, lo que facilita la inserción y eliminación de elementos sin desperdiciar memoria.

La estructura del historial se definió de la siguiente manera:

Listing 1: Estructura Historial

```
typedef struct _nodo_historial* Ptr_nodo_historial;
typedef Ptr_nodo_historial Stack;

typedef struct _nodo_historial
{
    Accion accion;
    Ptr_nodo_historial siguiente;
} Nodo_historial;
```

Adicionalmente, se menciona la estructura `Accion`, que es utilizada por distintos módulos y principalmente por el módulo de historial, ya que es uno de los atributos de cada nodo.

Listing 2: Estructura Accion

```
typedef struct _accion
{
    int tipo;
    char* nombre;
    char* desc;
    Objeto objeto;
} Accion;
```

El campo `tipo` identifica la clase de acción registrada, `desc` almacena el mensaje descriptivo y `objeto` guarda una copia del objeto involucrado, que es lo que permite restaurarlo al deshacer. En las acciones que no involucran un objeto, este campo queda inicializado en cero.

Las pilas tienen definidas operaciones básicas, siendo estas:

- makeEmpty: crea el nodo centinela.
- push: agrega x al extremo de la pila (top).
- pop: elimina el elemento que se encuentra en el top.
- isEmpty: indica si la pila tiene elementos.
- top: entrega el valor del extremo, sin retirarlo.

Para la implementación del historial, al ser la única estructura que utiliza pilas, estas funciones fueron renombradas como:

- makeEmpty → `historial_inicializar()`
- push → `historial_push(Accion accion, Stack s)`
- pop → `historial_pop(Stack s)`
- isEmpty → `historial_vacio(Stack s)`
- top → `historial_top(Stack s)`

A continuación se describe cada función implementada en el módulo de historial, junto con las decisiones de diseño que las motivaron.

historial_inicializar (makeEmpty). Crea el nodo centinela de la pila y retorna un puntero a él.

Listing 3: Función historial_inicializar

```
Stack historial_inicializar(){
    Stack s = calloc(1, sizeof(Nodo_historial));
    if (s == NULL){
        printf("Error: Falta de memoria\n");
        return NULL;
    }
    s->siguiente = NULL;
    return s;
}
```

Se optó por un **nodo centinela** (cabecera) que no almacena datos y cuyo campo `siguiente` apunta al tope de la pila. De esta forma, `push` y `pop` no necesitan tratar de forma especial la pila vacía, y el puntero `Stack` entregado a las demás funciones nunca cambia de valor, lo que evita tener que usar punteros dobles. Se utiliza `calloc` en lugar de `malloc` para que todos los campos del nodo comiencen en cero. Con `malloc`, el campo `siguiente` podía quedar con un valor sin inicializar, lo que provocó un error detectado con Valgrind al recorrer la pila durante el guardado de la partida.

historial_vacio (isEmpty). Indica si la pila no contiene acciones.

Listing 4: Función historial_vacio

```
int historial_vacio(Stack s){
    if(s->siguiente == NULL)
        return 1;
    return 0;
}
```

Retorna 1 si el centinela no tiene un nodo siguiente y 0 en caso contrario. Es la función de la que dependen las demás para evitar operar sobre una pila sin elementos.

historial_push. Agrega una acción al tope de la pila.

Listing 5: Función historial_push

```
Stack historial_push(Accion accion, Stack s){
    Ptr_nodo_historial aux_historial;
    aux_historial = malloc(sizeof(Nodo_historial));
    if (aux_historial == NULL){
        printf("Error: Falta de memoria\n");
        return s;
    }
    aux_historial->accion = accion;
    aux_historial->siguiente = s->siguiente;
    s->siguiente = aux_historial;
    return s;
}
```

Reserva un nodo nuevo, copia en él la acción recibida y lo enlaza inmediatamente después del centinela, de modo que pase a ser el nuevo tope. Si falla la reserva de memoria, se informa el error y la pila queda intacta.

historial_pop. Elimina la acción que se encuentra en el tope y libera su memoria.

Listing 6: Función historial_pop

```
Stack historial_pop(Stack s){
    if(historial_vacio(s)){
        printf("Error: historial vacio.\n");
        return s;
    }
    Ptr_nodo_historial primero = s->siguiente;
    s->siguiente = primero->siguiente;
    free(primero->accion.desc);
    free(primero->accion.nombre);
    free(primero->accion.objeto.nombre);
    free(primero);
    return s;
}
```

Primero verifica que la pila no esté vacía. Luego desenlaza el nodo del tope y libera, en orden, los campos dinámicos de la acción (descripción, nombre y nombre del objeto)

y finalmente el nodo. Liberar los campos internos antes que el nodo es necesario porque, de lo contrario, se perdería la referencia a esa memoria y se producirían fugas. Dado que `free(NULL)` es una operación segura, no es necesario comprobar cuáles campos fueron asignados.

historial_top. Entrega la acción del tope sin retirarla.

Listing 7: Función historial_top

```
Accion* historial_top(Stack s){
    if (s == NULL || historial_vacio(s))
        return NULL;
    return &(s->siguiente->accion);
}
```

Retorna un **puntero** a la acción y no una copia. Esta decisión permite que otros módulos modifiquen la acción recién registrada; por ejemplo, el módulo de aventura le asigna el objeto descartado y el de combate le asigna la copia del objeto utilizado. Si la pila está vacía o no está inicializada retorna `NULL`, por lo que quien la llame debe comprobar el resultado.

historial_registrar_accion. Función de conveniencia que construye una acción y la apila.

Listing 8: Función historial_registrar_accion

```
Stack historial_registrar_accion(Stack s, char* desc, int tipo){
    Accion nueva = {0};
    nueva.tipo = tipo;
    nueva.desc = malloc(strlen(desc) + 1);
    if (nueva.desc == NULL){
        printf("Error: Falta de memoria\n");
        return s;
    }
    strcpy(nueva.desc, desc);
    historial_push(nueva, s);
    return s;
}
```

Los demás módulos solo necesitan indicar un mensaje y un tipo, sin conocer la estructura interna de `Accion`. La acción se inicializa en cero ({0}) para que los campos `nombre` y `objeto` queden en `NULL` cuando no se usan. Esto es importante porque `historial_pop` libera todos los campos sin distinguir el tipo de acción. La descripción se copia en memoria propia, de modo que el historial no dependa de que la cadena original siga existiendo.

historial_deshacer_accion. Es la función que implementa el requerimiento de deshacer la última acción.

Listing 9: Función historial_deshacer_accion

```
Stack historial_deshacer_accion(Stack s, Inventario inv, Jugador*
    jugador){
    Accion *top = historial_top(s);
```

```
3       if (top == NULL){
4           printf("Error: historial vacio\n");
5           return s;
6       }
7       if(top->tipo == 7 && top->objeto.nombre != NULL){
8           if (top->objeto.tipo == 1)
9               jugador->ataque  -= top->objeto.valor_efecto;
10          else if (top->objeto.tipo == 3)
11              jugador->defensa -= top->objeto.valor_efecto;
12          Insertar_Objeto_en_el_Inventario(inv, top->objeto);
13          top->objeto.nombre = NULL;
14          printf("Se deshizo la accion. Recuperas el objeto
                utilizado\n");
15          historial_pop(s);
16      }
17      else
18          printf("No es posible deshacer la ultima accion\n");
19      return s;
20  }
```

El funcionamiento es el siguiente:

1. Consulta el tope con `historial_top`. Si la pila está vacía, informa el error.
2. Solo actúa si la acción es de tipo 7 (utilizar objeto) y tiene un objeto asociado. Cualquier otro tipo de acción no se puede deshacer, tal como lo establece el enunciado.
3. Revierte el efecto del objeto sobre el jugador: resta el bonus de ataque (objetos tipo 1) o de defensa (objetos tipo 3).
4. Devuelve el objeto al inventario con `Insertar_Objeto_en_el_Inventario`.
5. Asigna `NULL` a `objeto.nombre` y recién entonces llama a `historial_pop`.

Las decisiones más relevantes son:

- **Copia profunda del objeto.** Al usar un objeto, el inventario libera su memoria. Por eso, el módulo de combate guarda en la acción una copia independiente (con su propio nombre) antes de eliminarlo del inventario. Sin esa copia, el historial no tendría nada que restaurar.
- **Transferencia de propiedad.** Al reinsertar el objeto, el inventario pasa a ser dueño de la cadena `nombre`. Por ello se asigna `NULL` antes del `pop`: si no se hiciera, `historial_pop` liberaría esa memoria y el inventario quedaría con un puntero inválido (*use-after-free* y doble liberación).
- **Tipo de acción propio.** Se asignó el tipo 7 exclusivamente al uso de objetos para distinguirlo del descarte (tipo 6). Así, la condición de deshacer es explícita y no se activa por error con otras acciones.
- **Efecto de las pociones.** Las pociones (objetos de tipo 2) no revierten la vida recuperada, ya que el valor previo no se almacena. Solo se restituye el objeto. Si bien deberia de ser revertido al ser considerado uso de objeto, se atribuye esto a un error de diseño nuestro.

historial_print. Muestra por pantalla todas las acciones registradas.

Listing 10: Función historial_print

```
void historial_print(Stack s){
    if (s == NULL){
        printf("Error: historial no inicializado\n");
        return;
    }
    if (historial_vacio(s)){
        printf("El historial esta vacio.\n");
        return;
    }
    Ptr_nodo_historial nodo = s->siguiente;
    printf("-------------- Historial -------------\n");
    int i = 1;
    while (nodo != NULL){
        printf("%2d.\tTipo: %d\tDescripcion: %s\n",
               i, nodo->accion.tipo, nodo->accion.desc);
        nodo = nodo->siguiente;
        i++;
    }
}
```

Recorre la lista desde el tope hasta el final, por lo que las acciones se muestran de la más reciente a la más antigua. Esta función no modifica la pila: usa un puntero auxiliar para recorrerla y no retira elementos. Valida antes que la pila exista y no esté vacía.

historial_liberar. Libera por completo la memoria del historial al terminar el programa.

Listing 11: Función historial_liberar

```
void historial_liberar(Stack s){
    while (!historial_vacio(s))
        historial_pop(s);
    free(s);
}
```

4.3. Implementaciion de colas: modulo combate

Para el sistema de combate del juego, se requería que el jugador se enfrentara a un grupo de enemigos siguiendo un estricto orden de aparición. Por esta razón, la estructura de datos elegida fue una cola. Esta decisión de diseño es la ideal para este caso, ya que permite que los enemigos se formen en una fila y el personaje principal deba derrotar obligatoriamente al monstruo que está al frente antes de poder avanzar al siguiente.

En el código, esta cola se construyó conectando nodos, manteniendo un control exacto sobre quién es el primer y el último enemigo de la fila. Cuando el jugador explora una zona y se genera un encuentro, el sistema utiliza la función encolar para crear a los rivales según la dificultad del área y los agrega al final de la fila, pidiéndole al computador la memoria exacta para guardar los datos de cada enemigo, como su nombre, puntos de vida y daño de ataque.

Una vez que estalla la batalla a través de la función iniciar combate, el programa utiliza la operación desencolar para extraer al primer enemigo de la fila y ponerlo frente al jugador. Durante el enfrentamiento, el usuario interactúa mediante un menú por terminal donde puede decidir si realizar un ataque básico, revisar su inventario o utilizar un objeto específico para recuperar vida o mejorar su defensa. Si los puntos de vida del enemigo actual llegan a cero, este es derrotado de forma definitiva; luego, el sistema revisa cuántos enemigos quedan en espera y saca automáticamente al siguiente de la cola para continuar la pelea. Este ciclo de combate se repite de manera continua hasta que la fila de enemigos queda completamente vacía o hasta que los puntos de vida del personaje principal se agotan por completo.

4.4. Implementacion del modulo aventura

El modulo de aventura se encarga de la exploracion de las zonas y de los eventos que ocurren mientras el personaje avanza. No define una estructura de datos propia: su funcion es conectar los demas modulos, ya que durante una exploracion utiliza el inventario para guardar objetos, la cola de enemigos para iniciar combates, el historial para registrar lo ocurrido y el personaje, cuyos atributos pueden cambiar durante la partida. A continuacion se describen sus funciones.

explorar_zona. Controla el recorrido del personaje por una zona y decide que evento ocurre.

Listing 12: Funcion explorar_zona

```
void explorar_zona(int N, Inventario inv, int zona, Stack s,
    Jugador *jugador)
{
    int pasos = 0, continuar;

    while(pasos < N)
    {
        pasos++;

        if (pasos == 1)
            printf("Caminas␣entre␣los␣arboles.␣No␣encuentras␣nada
                .\n");
        else if (pasos == 2)
            printf("Sigues␣por␣el␣sendero.␣No␣encuentras␣nada.\n"
                );
        else if (pasos == 3)
            printf("Cruzas␣un␣pequeno␣arroyo.␣No␣encuentras␣nada
                .\n");
        else
            printf("No␣encuentras␣nada.\n");

        if(pasos == N)
        {
            int evento = rand()%3+1;

            switch (evento)
            {
                case 1:
                    obtener_objetos(inv, s);
                    break;
                case 2:
                    cola_enemigo cola_combate;
                    inicializar_cola(&cola_combate);
                    generar_enemigos_zona(&cola_combate, zona);
                    iniciar_combate(jugador, inv, &cola_combate,
                        s);
                    liberar_cola(&cola_combate);
                    break;
```

```
34                  case 3:
35                      printf("No encuentras nada\n");
36                      break;
37              }
38
39              if (jugador->vida <= 0)
40                  break;
41
42              /* Se pregunta si desea seguir explorando (1 o 2) */
43              if(continuar == 1)
44                  pasos = 0;
45              else
46                  break;
47          }
48      }
49  }
```

La funcion recibe el numero de pasos `N` que deben transcurrir antes de que ocurra un evento (en el juego, 3), el inventario, la zona elegida, el historial y un puntero al jugador. En cada vuelta del ciclo se suma un paso y se muestra un mensaje de recorrido sin hallazgos; al llegar al paso `N` se genera un evento aleatorio con `rand()`, que puede ser:

1. **Encontrar un objeto:** se llama a `obtener_objetos`.
2. **Encontrar enemigos:** se crea una cola de combate, se llena con enemigos segun la dificultad de la zona, se inicia el combate y al terminar se libera la cola.
3. **No ocurre nada:** se informa al jugador y se continua.

Despues del evento se verifica la vida del personaje: si llego a cero, se termina la exploracion de inmediato. En caso contrario, se pregunta si desea seguir explorando; si responde que si, el contador de pasos vuelve a cero y comienza un nuevo recorrido, y si responde que no, la funcion termina.

Las decisiones mas relevantes fueron:

- **Evento solo al completar los pasos.** Los mensajes de recorrido hacen que la exploracion se sienta progresiva, como pedia el enunciado, en lugar de que el evento ocurra de inmediato.
- **Parametro `N`.** El numero de pasos se recibe como parametro y no se fija dentro de la funcion, de modo que se pueda cambiar la duracion del recorrido sin modificar su logica.
- **Cola de enemigos local.** La cola se crea dentro del evento de combate y se libera inmediatamente despues. Asi existe solo mientras dura la pelea y no puede quedar memoria reservada entre exploraciones.
- **Validacion de la entrada.** La pregunta de continuar se repite hasta recibir un valor valido, y se limpia el buffer del teclado para que una entrada incorrecta no se arrastre a la siguiente lectura.
- **Corte por muerte.** Si el personaje muere en el combate, no tiene sentido preguntar si desea seguir, por lo que se rompe el ciclo antes de consultarlo.

obtener_objetos. Genera un objeto aleatorio y deja que el jugador decida si lo conserva o lo descarta.

Listing 13: Funcion obtener_objetos

```
Objeto obtener_objetos(Inventario inv, Stack s)
{
    int probabilidad = (rand() % 100) + 1;
    Objeto obj = datos_objetos(probabilidad);
    int opcion = 0;

    printf("Encontraste un objeto en tu trayecto.\n");
    Mostrar_Objeto(obj);
    printf("Deseas conservarlo? 1.Si 2.No\n");

    /* Se repite la lectura hasta recibir 1 o 2 */

    if(opcion == 1){
        printf("Guardas el objeto...\n");
        Insertar_Objeto_en_el_Inventario(inv, obj);
    }
    else
    {
        historial_registrar_accion(s, "Se ha descartado un objeto
           ", 6);
        Accion* a = historial_top(s);
        if (a != NULL && a->tipo == 6)
            a->objeto = obj;
        else
            Borrar_Objeto(obj);
    }
    return obj;
}
```

Primero se obtiene un numero entre 1 y 100 y se lo entrega a `datos_objetos`, que segun el rango devuelve un objeto distinto. Asi, los objetos mas comunes (como la espada corta de cobre) tienen mayor probabilidad de aparecer que los mas raros (como la cota de malla). Luego se muestra el objeto y se pide una decision:

- Si el jugador lo conserva, se inserta en el inventario.
- Si lo descarta, se registra la accion en el historial (tipo 6) y se guarda el objeto dentro de ella.

Las decisiones de diseno fueron:

- **Probabilidades por rangos.** Usar un unico numero aleatorio dividido en rangos permite controlar facilmente la rareza de cada objeto, ajustando solo los limites de la funcion `datos_objetos`.
- **El historial se queda con el objeto descartado.** En lugar de liberarlo de inmediato, la accion lo conserva, de modo que la memoria sigue teniendo un unico dueno y se libera con `historial_pop` cuando la accion sale de la pila.

- **Liberacion en caso de fallo.** Si no se pudo registrar la accion (por falta de memoria), el objeto se libera en el momento con `Borrar_Objeto`, para no perder la referencia a su memoria.
- **Retorno del objeto.** La funcion devuelve el objeto generado para que el modulo que la llame pueda usarlo si lo necesita. En el juego actual, el valor de retorno no se utiliza.

4.5. Implementacion del modulo interfaz

El modulo de interfaz es el punto de entrada del juego. Su responsabilidad es crear los elementos principales de la partida, mostrar el menu al jugador, llamar al modulo que corresponda segun la opcion elegida y liberar toda la memoria al terminar. Al ser quien coordina a los demas, no implementa ninguna estructura de datos; solo las utiliza. La funcion `main` se limita a llamar a `iniciar_juego`, de modo que toda la logica queda dentro de este modulo.

limpiar_buffer. Descarta los caracteres pendientes en la entrada del teclado.

Listing 14: Funcion limpiar_buffer

```
void limpiar_buffer()
{
    while(getchar() != '\n');
}
```

Cuando el jugador escribe algo que no es un numero, `scanf` falla y deja esos caracteres en el buffer, lo que provocaria que la siguiente lectura fallara de nuevo y el menu se repitiera sin fin. Esta funcion consume todo lo pendiente hasta el salto de linea. Se definio como funcion aparte para reutilizarla cada vez que una lectura falla, en lugar de repetir el mismo ciclo en varios lugares.

iniciar_juego. Contiene el ciclo principal del juego.

Listing 15: Funcion iniciar_juego (resumida)

```
void iniciar_juego()
{
    Jugador jugador = jugador_crear("Aria");
    Inventario inventario = Crear_inventario_vacio(NULL);
    Stack historial = historial_inicializar();

    if (!cargar_partida(&jugador, inventario, historial, "partida
        .txt"))
        printf("Iniciando una nueva partida con el personaje %s.\
            n",
                jugador.nombre);

    int opcion = 0;
    while (opcion != 5)
    {
        /* Se muestra el menu y se lee la opcion */
```

```
15          switch (opcion)
16          {
17              case 1: /* Ver personaje */
18              case 2: /* Explorar zona */
19              case 3: /* Inventario */
20              case 4: /* Historial */
21              case 5: /* Guardar y salir */
22              default: /* Opcion no valida */
23          }
24      }
25
26      Destruir_Inventario(inventario);
27      historial_liberar(historial);
28      if (jugador.nombre != NULL) free(jugador.nombre);
29  }
```

La funcion trabaja en tres etapas.

Inicializacion. Se crea el personaje (llamado Aria), el inventario vacio y el historial. Luego se intenta cargar una partida guardada desde `partida.txt`. Si el archivo no existe, se informa que se inicia una partida nueva con los valores por defecto.

Ciclo del menu. Se repite hasta que el jugador elige la opcion 5 o el personaje muere. Cada opcion llama al modulo correspondiente:

- **1. Ver personaje:** muestra nombre, vida, ataque y defensa con `jugador_print`.
- **2. Explorar zona:** muestra las tres zonas, valida la eleccion, inicializa el generador de numeros aleatorios y llama a `explorar_zona`. Si al volver el personaje murio, muestra el fin del juego junto con el resumen de la aventura (el historial) y termina el ciclo.
- **3. Inventario:** muestra los objetos con `Mostrar_Inventario`.
- **4. Historial:** muestra todas las acciones registradas.
- **5. Guardar y salir:** guarda la partida con `guardar_partida` y termina el ciclo.

Despues de cada opcion se registra una accion en el historial con un mensaje y un tipo distinto, para que el resumen final refleje lo que hizo el jugador.

Liberacion de memoria. Al salir del ciclo se destruye el inventario, se libera el historial y se libera el nombre del personaje, que fue reservado dinamicamente. De esta forma el programa termina sin dejar memoria reservada.

Las decisiones de diseno mas relevantes fueron:

- **Un solo ciclo con `switch`.** Un menu en un ciclo hasta elegir la opcion de salida es simple de entender y de ampliar: agregar una opcion nueva solo requiere un nuevo caso.
- **Validacion de la entrada.** Si el jugador escribe algo que no es un numero, se informa el error, se limpia el buffer y se vuelve a mostrar el menu, evitando que el programa se detenga o entre en un ciclo infinito.
- **Carga automatica al iniciar.** Al intentar siempre cargar `partida.txt`, el jugador retoma su progreso sin tener que elegirlo en el menu.

- **Muerte como fin de partida.** Cuando la vida llega a cero se muestra el historial como resumen y se sale del ciclo, lo que le da un cierre claro a la aventura.
- **Guardado antes de registrar la salida.** La accion "Guardar y salir" se registra despues de guardar, por lo que ya no queda incluida en el archivo.
- **Aleatoriedad.** El generador se inicializa con la hora actual cada vez que se explora una zona, para que los eventos y enemigos cambien entre ejecuciones.

4.6. Implementación del módulo personajes

El módulo de personajes define al protagonista del juego y las operaciones básicas para crearlo y mostrar su estado. Al igual que los módulos de aventura e interfaz, no implementa una estructura de datos de las vistas en la asignatura, sino que define una estructura propia que reúne los atributos del personaje, los cuales son consultados y modificados por los demás módulos durante la partida: el combate resta vida y suma ataque o defensa, el historial revierte esos bonus al deshacer y el módulo de archivos los guarda y los carga.

La estructura `Jugador` se definió de la siguiente manera:

Listing 16: Estructura Jugador

```
typedef struct _jugador
{
    char* nombre;
    int vida;
    int ataque;
    int defensa;
} Jugador;
```

El nombre se almacena como un puntero a memoria dinámica y no como un arreglo de tamaño fijo, de modo que se reserve únicamente el espacio que el nombre necesita, en la misma línea del resto del proyecto. Los demás atributos son enteros: `vida` indica los puntos de vida actuales, `ataque` el daño que se inflige a un enemigo y `defensa` la cantidad de daño que se reduce de cada ataque recibido. A continuación se describen sus funciones.

jugador_crear. Crea un personaje con atributos por defecto y el nombre recibido como parámetro.

Listing 17: Función jugador_crear

```
Jugador jugador_crear(char* nombre){
    Jugador jugador;

    jugador.nombre = malloc(strlen(nombre) + 1);
    if(jugador.nombre == NULL){
        printf("Error: No hay memoria disponible para crear
           nombre del personaje\n");
        return jugador;
    }

    strcpy(jugador.nombre, nombre);

```

```
12      jugador.ataque = 5;
13      jugador.defensa = 3;
14      jugador.vida = 5;
15
16      return jugador;
17  }
```

La función reserva con `malloc` exactamente

5. Problemas que se dieron en el proyecto

Al principio, la exploración del mapa estaba desconectada del menú principal y no mostraba los mensajes paso a paso que pedía el manual. Para arreglarlo, se conectaron las funciones de aventura con la interfaz principal y se ajustaron los textos para que el personaje caminara y encontrara cosas como en el ejemplo del proyecto.

Otro problema fue que el historial no estaba registrando lo que pasaba en las peleas. Se soluciono pasándole la variable del historial a la función de combate. Así, el juego anota cuando derrotas a un enemigo y usa ciclos especiales para escribir y leer esos datos del archivo de texto.

Al hacer esos cambios con el historial, empezaron los errores de compilación porque el programa no reconocía el tipo de dato Stack. Esto se reparo incluyendo la cabecera correcta en el archivo.

También se notó que el juego siempre entregaba un objeto falso de prueba en lugar de los ítems reales. Para que funcionara bien, se movió el catálogo de objetos al archivo correcto y programamos una probabilidad al azar que entrega las espadas o pociones de verdad. Además, como cada objeto es único, se le puso un contador interno para que nunca se repitan los números de identificación en el inventario.

Otro problema fue de las armas en los combates, ya que al equiparlas no hacían daño extra y solo reemplazaban la fuerza base. Se cambió la fórmula para que el poder del arma se sume a tu ataque original.

Un error de diseño que se mencionó anteriormente: cuando se deshace una acción se pueden restaurar el ataque o defensa a como estaba antes, sin embargo, cuando el jugador decide usar un objeto de curacion el objeto se restaura perfectamente, pero no la vida, debido a que en diseño no se pensó en una situacion donde guardasemos la vida. Atribuimos esto a un error de diseño de; cual nos dimos cuenta en una etapa tardía del proyecto.

6. Pruebas de ejecucion y resultados

Pruebas de ejecucion:

Nosotros hicimos varias pruebas individuales a lo largo del proyecto, las cuales nos lanzaban distintos errores que fuimos corrigiendo a medida que avanzabamos en proyecto y en el cual se puede visualizar en los problemas que tuvimos.

La prueba final fue hecha con Claude Code la cual dio como resultado que compila sin warmings, sin errores y sin fugas de memoria. Las estructuras estan bien hechas, lo que es la pila, la cola y las listas enlazadas.

Resultados: Dentro de los resultados se puede apreciar un funcionamiento que puede dar rumbo a una gran cantidad de posibles rutas segun el orden de acciones a tomar, teniendo en cuenta esto uno puede diversificar bastante una unica partida, pudiendo llegar a tener una ruta bastante extensa, y asi mismo puede durar poco.

Esto debido a la funcionalidad de encontrarse con un evento, ya que uno puede encontrar un objeto que puede variar entre 5, o una horda de enemigos, y con esa probabilidad uno puede simplemente no tener ningun resultado bueno, asi como puede tener un resultado muy positivo.

7. Conclusiones

Al finalizar el proyecto se logró desarrollar un juego de rol funcional en lenguaje C que se ejecuta por terminal, en el cual el personaje puede explorar tres zonas de distinta dificultad, encontrar objetos, administrar un inventario, enfrentarse a grupos de enemigos y guardar o cargar su progreso desde un archivo de texto. Con esto se cumplió el objetivo general de construir y aplicar listas enlazadas, pilas y colas para gestionar las distintas partes del juego.

Cada estructura de datos se eligió según lo que necesitaba el módulo que la usaba. El inventario se implementó con una lista enlazada simple con nodo centinela, lo que permitió agregar, buscar y eliminar objetos de forma dinámica sin definir un tamaño máximo. El combate se resolvió con una cola, ya que los enemigos debían enfrentarse en el mismo orden en que aparecían, lo que se ajustó exactamente al principio *First In, First Out*. Por último, el historial se implementó con una pila, porque la acción que interesa consultar o deshacer es siempre la más reciente, lo que coincide con el principio *Last In, First Out*. Esto mostró que elegir bien la estructura simplifica la lógica del programa y evita soluciones más complicadas de lo necesario.

Uno de los aprendizajes más importantes fue el manejo de la memoria dinámica. Durante el desarrollo aparecieron errores que no eran evidentes a simple vista, como el uso de un valor sin inicializar en el nodo centinela del historial, detectado con Valgrind, o el cuidado que hubo que tener al deshacer una acción: el nombre del objeto cambia de dueño al volver al inventario, por lo que debía dejar de ser liberado por el historial. Resolver estos casos permitió comprender mejor quién es responsable de cada bloque de memoria y por qué es necesario liberar los campos internos de una estructura antes que la estructura misma.

También se presentaron dificultades de integración entre módulos, como conectar la exploración con el menú principal, pasar el historial a las funciones de combate para registrar lo que ocurría en las peleas, o corregir el comportamiento de las armas para que su poder se sumara al ataque base del jugador. Estas correcciones mostraron que cada módulo puede funcionar bien por separado y aun así fallar al unirse con los demás, por lo que el trabajo en equipo y las pruebas fueron fundamentales.

En cuanto a las limitaciones, la principal es que al deshacer el uso de una poción el objeto se recupera, pero la vida del personaje no vuelve a su valor anterior, porque no se guardó la vida previa en el historial. Este es un error de diseño que se detectó en una etapa tardía del proyecto. Como mejora, la estructura `Accion` podría incluir un campo adicional para almacenar el estado del jugador antes de usar el objeto, de modo que cualquier acción pudiera revertirse por completo.

En resumen, el proyecto permitió aplicar en un caso concreto los conceptos vistos en la asignatura, reforzar el trabajo con punteros y memoria dinámica, y comprender cómo se combinan distintas estructuras de datos dentro de un mismo programa.

8. Referencias

[1] J. Aldridge Águila, .Estructura de datos: Listas enlazadas,"material de clase, Universidad de Magallanes (UMAG), Punta Arenas, Chile, sep. 2026.

[2] J. Aldridge Águila, .Estructura de datos: Listas enlazadas - operaciones,"material de clase, Universidad de Magallanes (UMAG), Punta Arenas, Chile, sep. 2026.

[3] J. Aldridge Águila, .Estructura de datos: Listas enlazadas dobles y circulares,"material de clase, Universidad de Magallanes (UMAG), Punta Arenas, Chile, sep. 2026.

[4] J. Aldridge Águila, .Estructura de datos: Pilas y colas,"material de clase, Universidad de Magallanes (UMAG), Punta Arenas, Chile, sep. 2026.

9. Anexos

9.1. Contribucion de cada estudiante

- En conjunto: README, pruebas, informe y presentacion

- Eban: inventario.c y objetos.c. Se encargará de programar la lógica base de los ítems en objetos.c y de construir el sistema de catálogo en inventario.c para agregar, buscar, equipar y eliminar objetos.

- Yanira: historial.c y personajes.c. Se encarga de la responsabilidad central es historial.c, donde registrará los eventos y validará la función de extraer solo para deshacer objetos descartados. Además se encarga de personajes.c, que gestiona los atributos (vida, ataque, defensa) y el estado actual del protagonista.

- Martina: combate.c y archivos.c. se encarga de desarrollar combate.c forzando el orden de enfrentamiento de los enemigos según su aparición. Además, se encarga de archivos.c, construyendo la lógica para escribir y guardar el progreso del juego en un archivo y cargarlo al reiniciar.

9.2. Declaracion de IA

Declaramos que sí usamos inteligencia artificial en este proyecto para probar el código y encontrar errores que eran difíciles de ver, tal como lo permiten las reglas del trabajo.

Por un lado, usamos la herramienta Claude Code únicamente para crear pruebas automáticas. Esta herramienta nos ayudó a armar pruebas rápidas para revisar que nuestras listas, pilas y colas funcionaran sin problemas, lo que nos permitió comprobar que el juego no fallara en situaciones difíciles (como cuando una lista se queda vacía o hay falta de memoria) y corregir detalles antes de la entrega.

Por otro lado, la utilizamos para, buscar errores en el código cuando el programa fallaba, se cerraba o tenía problemas de memoria, nos ayudó a encontrar dónde estaba el error y cómo solucionarlo, ademas de verificar si el codigo no se estaba desviando de lo pedido en la rubrica.

9.3. Declaración de IA: Declaración individual

- **Eban Delgado**: Implemente IA para depurar el codigo, con el objetivvo de encontrar errores que dificultaran el correcto funcionamiento y que se acple a la rubrica correctamente sin desviarse, y tambien para encontrar errores al momento de desarrollar el informe, con el objetivo de encontrar errores de latex.

- **Yanira Mansilla**: Utilicé IA para depurar código, buscar errores y corregir fugas de memoria. Una de las correcciones que me entregó la IA (Claude) fue usar `calloc` en lugar de `malloc` en la función `historial_inicializar`. Esto se debió a que, en uno de los varios intentos anteriores a las pruebas finales, al ejecutar el programa con Valgrind apareció un error que indicaba que se usaba un valor sin inicializar. Claude propuso utilizar `calloc`, que reserva la memoria y deja todos sus campos en cero, de modo que el puntero `siguiente` del nodo centinela comienza siempre en `NULL`.

- **Martina Márquez**: Se utilizo IA para crear pruebas automatizadas con Claude Code, en la parte final, para observar si se nos paso algun detalle en las pruebas que hicimos por nuestra cuenta, ademas se uso la IA para encontrar errores no visibles o fugas de memoria.